

Plano — Execução de Agentes no LangNet

Data: 25 de setembro de 2026 **Origem:** análise do executor que roda em 192.168.1.116 (`valep12/visualtasksexec/tropicalsales`), dirigido de fora com gravação quadro a quadro da conversa entre a tela e o servidor de agentes.

Resumo em uma página

O framework tem **dois lados**: o LangNet é a **fábrica genérica** (transforma requisitos em agentes, tarefas e uma rede de Petri executável, para **qualquer** domínio) e o `visualtasksexec` é a **máquina** que roda o que a fábrica produziu. TropicalSales, BioByte, Uso do Solo, Quântica e ClinIA são **instâncias de teste da fábrica** — nenhuma delas é o produto. O elo entre eles é a rede de Petri gravada no banco — ela **é** o projeto.

O lado da máquina funciona e foi observado funcionando: cadeia completa em **1 minuto e 20 segundos**, quatro agentes em sequência, lendo e-mails reais, classificando e redigindo resposta.

O lado da fábrica **gera a rede corretamente, mas nunca conseguiu executá-la**, por um defeito de uma linha. Além disso, a aplicação final que a fábrica gera **ignora a rede** e chama tarefas soltas, uma por vez — toda a orquestração fica sem uso no produto entregue ao cliente.

E a verificação feita depois da primeira versão deste plano mudou o diagnóstico: a execução **nunca foi transferida** para o LangNet. Nove das quatorze peças do executor não existem no nosso repositório, e o nosso servidor de agentes **fala um protocolo diferente** — sem o handshake de abertura, sem o fluxo passo a passo e sem as etiquetas. Trazer a tela sem trazer o protocolo não funciona: são as duas pontas do mesmo contrato.

Este plano cobre: (1) o que está errado no nosso, (2) a anatomia completa da interface que vamos trazer, (3) **o contrato da execução, mensagem por mensagem, e a conferência do que foi transferido**, (4) como a bancada entra no LangNet, (5) o padrão de interface da aplicação final, e (6) o papel do back-end genérico e do banco.

Parte 1 — O que está errado no nosso

Defeito 1 — o executor do nosso front-end não sabe esperar [crítico]

O código que a nossa fábrica escreve para cada lugar da rede é inteiramente baseado em **espera**: espera o servidor de agentes responder, espera o lugar anterior produzir. Mas a peça do nosso front-end que executa esse código foi construída sobre um interpretador que **não aceita espera**.

O molde que mandamos para o app do cliente **tem** a correção. O nosso, não. É a correção que ficou pela metade quando aquela sessão morreu por falta de memória em junho.

Consequência: a nossa tela nunca conseguiu rodar a lógica que a nossa própria fábrica escreve. Nada executa. É por isso que a execução "não funciona" — não é o modelo, não é o prompt, não é a rede: é o interpretador.

Correção: uma linha, copiando o que já existe no molde.

Defeito 2 — ninguém pergunta se o lugar anterior terminou

A transição fica apta olhando **duas** coisas: se há token e se a condição passa. O executor de referência tem um **terceiro juiz**, num módulo separado de propósito, cuja única pergunta é: "todos os lugares que alimentam esta transição concluíram sua execução?".

No nosso, esse juiz não existe. Quem segura a fila é uma espera ocupada escrita dentro do código do lugar seguinte, com prazo de 90 segundos.

Defeito 3 — fracasso conta como sucesso

Essa espera decide "o anterior já produziu algo útil?" verificando se sobrou qualquer campo além de um punhado de campos de controle. Um lugar que **falhou** devolve o que recebeu **mais um aviso de erro** — e o aviso de erro conta como "algo útil".

Resultado: o erro atravessa a rede calado e a tarefa seguinte trabalha em cima de lixo.

Isto não é hipótese. Foi observado na máquina que "funciona bem": o agente gerador de respostas tentou responder o pedido 122047 (20 alças), a ferramenta de envio recusou por validação de argumentos, o agente registrou no próprio raciocínio que "a ferramenta exige cc e bcc" — **e a tarefa reportou concluída = verdadeiro**. O e-mail nunca saiu e a rede declarou sucesso.

Defeito 4 — as telas de negócio que geramos ignoram a rede

Hoje a aplicação final chama **uma tarefa por vez**, direto no servidor de agentes. Toda a orquestração que a fábrica produz — a sequência, os arcos, o estado que acumula, os pontos de espera — **não é usada pelo produto entregue**. Ela só roda na bancada de diagnóstico.

Este é o único item que não é conserto: é decisão de arquitetura. Está tratado na Parte 4.

Defeito 5 — a memória da execução existe e nunca foi escrita

O banco é **o mesmo** dos dois lados (mesmo servidor, mesma base). E ele já tem três tabelas desenhadas para guardar execução: sessões de execução, execuções de tarefa e saídas produzidas.

Estado real, medido:

tabela	para que serve	linhas
sessões de execução	uma linha por rodada da rede, com marcação inicial/final e contadores	158 — mas 0 com marcação de rede
execuções de tarefa	uma linha por tarefa, com entrada, saída e registro detalhado	0
saídas produzidas	documentos e arquivos gerados	0

Duas leituras:

1. **Nenhuma execução real jamais foi gravada.** O app fala direto com o servidor de agentes e nunca avisa o back-end. A auditoria existe no papel e está vazia.
2. **A tabela de sessões foi ocupada pelo nosso pipeline de geração** (124 linhas com documento de requisitos). Ela foi desenhada para execução de rede e nós passamos a usá-la para outra coisa. Há colisão de significado no mesmo lugar.

Parte 2 — Anatomia da interface que vamos trazer

Inventário completo do que existe em `tropicalsales/src`, com a função de cada elemento. O que está marcado **[morto]** não é renderizado e não deve ser copiado.

Moldura e entrada

elemento	linhas	função
<code>App.js</code>	8	raiz; renderiza a tela principal e nada mais
<code>MainExecutorTarefas.jsx</code>	248	a moldura: barra lateral recolhível, busca a lista de projetos no back-end, carrega o projeto escolhido e converte a rede recebida para o nome de campo que a tela espera
<code>ProjectSelector.jsx</code>	118	seletor alternativo de projeto [morto]
<code>GenericFrameworkPageV2.jsx</code>	84	tela alternativa com as abas-maquete [morto]
<code>tabs/OperationTab</code> , <code>InputsTab</code> , <code>OutputsTab</code> , <code>LogsTab</code>	26-32 cada	maquetes com o aviso "Aba em Desenvolvimento" [morto]
<code>tabs/ExecutionTab.jsx</code>	194	aba da tela alternativa [morto]

Atenção: as abas "não implementadas" que você mencionou são estas quatro maquetes — elas pertencem à tela alternativa que não roda. As abas **reais** estão todas dentro da tela grande, escritas em linha, e **funcionam**.

A tela

elemento	linhas	função
<code>ExecutorTarefasNew.jsx</code>	4.922	a tela inteira: cabeçalho do projeto, contadores, controles de execução, desenho da rede e as seis abas
<code>ExecutorTarefas.jsx</code>	23.625	versão anterior da mesma tela [morto]

As seis abas, em ordem, todas dentro deste arquivo:

aba	função
Operação	a principal — a rede desenhada com o token andando, os controles (Iniciar, Reiniciar, Contínua, Pausar por Tarefa, Modo Simulação) e o painel do simulador
Execução	foco na tarefa corrente: mostra as últimas entradas e as últimas saídas daquele lugar
Inputs	o que entrou em cada lugar, filtrado por tarefa
Outputs	o que saiu de cada lugar, com opção de editar o documento produzido e de limpar
Logs	o registro corrido da rodada
VerbosePanel V8	as etiquetas universais de cada tarefa (ver abaixo)

O estado que alimenta as abas é um único balde dividido em cinco gavetas — operação, entradas, execução, saídas, registro — e as mensagens que chegam do servidor de agentes são **distribuídas** por gaveta conforme o tipo. É esse roteamento que faz cada aba mostrar coisa diferente da mesma conversa.

Os dois painéis

1. Painel móvel "Acompanhamento de Execução" (`PetriNetExec/VerbosePanel.jsx` , 522 linhas)

É a janela flutuante que aparece **sozinha** assim que a execução começa, e fica **fora das abas** — você pode estar em qualquer aba que ela continua lá.

- arrastável pelo cabeçalho, com a posição guardada no navegador entre sessões
- minimiza para uma tarja fina
- o conteúdo é um documento vivo, em três seções — **entradas, execução, saídas** — desenhado no mesmo formato do relatório de teste
- rola sozinho conforme chegam passos novos
- **ponto de atenção:** os nomes das tarefas da Tropical estão **gravados no código** dele ("3. CHECK STOCK"). Para reaproveitar, tem de ser alimentado pela lista de tarefas do projeto.

2. Painel de etiquetas "V8" (`components/VerbosePanel.js` , 675 linhas)

Vive na sexta aba e mostra, para cada tarefa, **onze etiquetas**: nome da tarefa, nome do agente, entrada da tarefa, entrada da ferramenta, qual ferramenta usou, os passos que

seguir, a saída, o tipo da saída, o pensamento do agente, a saída da ferramenta e se concluiu.

É esta peça que transforma "deu erro" em "o agente chamou a ferramenta de envio sem cópia oculta e ela recusou". Foi olhando estas etiquetas que o defeito do e-mail apareceu. Sem elas, a bancada é enfeite.

O motor da rede

elemento	linhas	função
PetriNetSimulator.js	817	o motor: matrizes de pré e pós-condição, quem está apto, disparar, mover token
PlaceProcessor.js	800	executa o código de cada lugar quando o token chega, numa caixa fechada com utilitários — é aqui que mora a correção do Defeito 1
GuardEvaluator.js	232	avalia apenas a condição escrita na transição
LogicaPlacesConcluida.js	290	o terceiro juiz que nos falta: verifica se todos os lugares que alimentam a transição concluíram sua execução. Separado do avaliador de condição de propósito, para não misturar as duas responsabilidades
PetriNetEngine.js	463	fachada mais antiga do motor (estados, transições, disparo por conversa)
engine/ExecutionEngine.js	431	motor genérico de execução com ganchos por evento (lugar iniciou, concluiu, falhou; execução iniciou, concluiu, falhou)
PetriNetViewer.jsx	9.156	o desenho da rede — lugares, transições, arcos, faixas dos agentes, token
PetriNetEditor.jsx	9.392	o editor da rede (arrastar, criar, ligar)
SimulationPanel.jsx	750	o painel da direita: modo de execução, transições aptas, estado dos tokens, log de disparos
PetriPythonConverter.jsx	148	converte a rede para Python
Modal.jsx	56	janela auxiliar

A camada de conversa — a peça que eu não conhecia

elemento	linhas	função
utils/fakeWebSocket.js	174	intercepta a abertura de conversa feita pelo código de cada lugar. Apresenta a mesma interface de sempre, mas em vez de abrir uma ligação nova, redireciona para o cliente central
utils/centralWSClient.js	562	uma única ligação com o servidor de agentes, fila que serializa as tarefas, cache dos resultados de cada tarefa e repasse dos passos para o painel móvel
websocket/WebSocketV7Client.js	307	cliente direto, com reconexão e tentativas — usado fora do caminho da rede

Sem essas duas primeiras peças, cada lugar abriria a própria ligação: conversas duplicadas, resultados fora de ordem e nenhum cache para a tarefa seguinte montar a entrada. **Medido na observação: a rodada inteira usou uma ligação só.**

Estado e carregamento

elemento	linhas	função
<code>hooks/useProjectLoader.js</code>	205	carrega o projeto e sua rede a partir do back-end
<code>hooks/useContextState.js</code>	171	mantém o estado acumulado que atravessa a cadeia
<code>version.js</code>	4	versão da interface, exibida no cabeçalho

Parte 3 — O contrato da execução (o que faltava neste plano)

Esta parte é o coração. A execução **é** a rede de Petri: ao disparar, o token entra no lugar; o lugar tem um campo com um **script JavaScript**; o script é executado; ele abre a conversa com o servidor de agentes, manda a tarefa, espera, recebe e devolve o resultado como saída do lugar. Só então a transição seguinte dispara. **O que aparece nos painéis é literalmente o que volta pela conversa** — a tela não calcula nada, ela roteia mensagem por tipo.

Portanto: se o protocolo não for **idêntico**, a tela trazida da 116 não mostra nada.

3.1 — O protocolo, mensagem por mensagem

O que a tela envia (3 tipos):

mensagem	quando	conteúdo
<code>iniciar_execucao</code>	uma vez, antes de tudo	abre a rodada e limpa os arquivos da anterior
<code>execute_task</code>	uma por lugar	{ task_name, input_data }
<code>finalizar_execucao</code>	ao fim da rodada	fecha e consolida

`iniciar_execucao` **é obrigatório**. Sem ele o servidor recusa toda tarefa com "Execução não foi iniciada". Eu tropecei nisso e perdi tempo até descobrir.

O que o servidor devolve (8 tipos):

mensagem	quantas por rodada	para que serve
<code>welcome</code>	1	apresenta o projeto, a versão e a lista de tarefas suportadas

mensagem	quantas por rodada	para que serve
execucao_iniciada	1	confirma a abertura
execution_step	38 na rodada medida	o fluxo passo a passo — é o que enche o painel móvel e as abas
tags_extracted	1 por tarefa (4)	as onze etiquetas consolidadas da tarefa
task_completed	1 por tarefa	o resultado final daquela tarefa
task_error	em falha	erro da tarefa
error	em falha de protocolo	erro geral
execucao_finalizada	1	encerra

O envelope da resposta final — campo por campo, exatamente:

```
{ "type": "task_completed",
  "data": { "task_name", "success", "duration", "adapter_version",
            "enhanced_parser", "universal_tags", "result", "timestamp" } }
```

O envelope de erro:

```
{ "type": "task_error",
  "data": { "task_name", "success": false, "error",
            "adapter_version", "enhanced_parser", "timestamp" } }
```

3.2 — As entradas e saídas entre lugares

O primeiro lugar recebe só os parâmetros do fluxo, seis campos: `max_emails`, `imap_server`, `timeout_seconds`, mais três de controle da rede (`from_transition`, `received_at`, `tokens_received`).

Do segundo lugar em diante, cada um recebe **o envelope inteiro do resultado anterior**, com estes sete campos no primeiro nível: `task_name`, `timestamp`, `success`, `adapter_version`, `enhanced_parser`, `universal_tags`, `backend_result`.

Quem desembulha é o tradutor do lado do servidor, não o código do lugar. Ele cava dentro de `backend_result` e entrega ao motor Python um pacote simples — na prática, um campo só com o JSON real. É por isso que existem as buscas em profundidade no tradutor.

3.3 — O estado padronizado que atravessa a cadeia

O tradutor mantém um **estado com forma fixa**, declarado uma vez por projeto. No Tropical são quinze campos: três de controle (`place_id`, `task_name`, `timestamp`), um parâmetro (`max_emails`), quatro documentos em texto (`emails_json`, `classified_json`, `stock_checked_json`, `response_report_md`), quatro em objeto (`emails_data`, `classification_data`, `stock_data`, `response_data`), mais `execution_log` e `current_task`.

Cada tarefa lê os campos que precisa e escreve o seu. É esta a padronização — e é ela que faz a acumulação funcionar. O contrato de "o que ler e o que escrever" está declarado **em prosa**, nos dois campos de sempre das tarefas: "devolva a mesma estrutura que recebeu e acrescente tais campos" / "você vai receber exatamente isso".

3.4 — As onze etiquetas

Emitidas ao fim de cada tarefa, no tipo `tags_extracted` :

`TASK_NAME` · `AGENT_NAME` · `TASK_INPUT` · `TOOL_INPUT` · `USED_TOOL` · `TASK_STEP` · `TASK_OUTPUT` · `TASK_OUTPUT_TYPE` · `AGENT_THOUGHT` · `TOOL_OUTPUT` · `TASK_COMPLETED`

São elas que transformam "deu erro" em "o agente chamou a ferramenta de envio sem cópia oculta e ela recusou". Sem elas, tanto a bancada quanto o painel de evidência da aplicação final ficam vazios.

Parte 3-B — Verificação: o que foi transferido para o LangNet

Conferência arquivo por arquivo e campo por campo. **Resultado: a execução nunca foi transferida.**

Frontend — 9 das 14 peças centrais não existem no nosso

peça	função	no nosso
<code>PlaceProcessor</code>	executa o código do lugar	existe (com o Defeito 1)
<code>GuardEvaluator</code>	condição da transição	existe
<code>SimulationPanel</code>	painel do simulador	existe
<code>PetriNetSimulator</code>	motor da rede	só no molde do app gerado
<code>LogicaPlacesConcluida</code>	juiz de "lugar concluiu"	não existe
<code>centralWSClient</code>	ligação única + fila + cache	não existe
<code>fakeWebSocket</code>	intercepta a conversa dos lugares	não existe
<code>VerbosePanel (móvel)</code>	acompanhamento flutuante	não existe
<code>PetriNetViewer</code>	o desenho da rede	não existe
<code>useContextState</code>	estado acumulado	não existe
<code>useProjectLoader</code>	carrega projeto e rede	não existe
<code>ExecutionEngine</code>	motor genérico com ganchos	não existe
<code>PetriNetEngine</code>	fachada do motor	não existe
<code>WebSocketV7Client</code>	cliente direto	não existe

Servidor — o nosso fala outro protocolo

Busca no nosso gerador pelos elementos do protocolo da 116:

elemento	ocorrências no nosso gerador
<code>iniciar_execucao</code> / <code>finalizar_execucao</code>	0
<code>execution_step</code>	0
<code>tags_extracted</code>	0
<code>universal_tags</code> / extrator de etiquetas	0
<code>context_state_list</code>	0
<code>convert_v1_to_backend_format</code>	0
<code>format_task_response</code> / <code>base_adapter</code>	0

O nosso servidor emite seis tipos — `connected`, `task_start`, `task_info`, `task_completed`, `error`, `pong` — e **nada entre o começo e o fim da tarefa**. Não há fluxo passo a passo e não há etiquetas.

E o envelope difere. O nosso: `{ type, timestamp, data:{ task_name, result } }`. O da 116: `{ type, data:{ task_name, success, duration, adapter_version, enhanced_parser, universal_tags, result, timestamp } }`.

Consequência prática

Se trouxermos a tela da 116 para o LangNet **hoje, sem mexer no servidor**, acontecem três coisas:

1. A tela manda `iniciar_execucao` e **o nosso servidor não conhece essa mensagem**.
2. Os painéis ficam **vazios**, porque ninguém emite o fluxo passo a passo.
3. A aba de etiquetas fica **vazia**, porque ninguém emite as etiquetas.

Trazer a tela sem trazer o protocolo não funciona. São as duas pontas do mesmo contrato.

E o que falta para TODA aplicação gerada pelo LangNet

Aqui está o item mais importante deste plano, e é preciso enunciá-lo sem ambiguidade: **o LangNet é uma fábrica genérica**. TropicalSales, BioByte, Uso do Solo, Quântica e ClinIA são **instâncias de teste** — existem para provar a fábrica, não são o produto. Tudo o que esta seção descreve vale para **todo projeto novo**, sem exceção.

O tradutor do Tropical é **escrito à mão para o Tropical**: o estado de quinze campos e as conversões têm um ramo por tarefa, com os nomes do domínio de e-mails dentro. Serve como **referência de forma**, nunca como código a copiar.

Portanto, o tradutor vira **etapa do pipeline**, emitida para cada projeto a partir dos artefatos que já produzimos — especificação de agentes e tarefas, sequência de tarefas e rede de Petri:

o que gerar	de onde sai
o estado padronizado do projeto (a forma fixa que atravessa a cadeia)	dos campos que cada tarefa declara ler e escrever, na especificação de agentes e tarefas
uma conversão de entrada por tarefa (desembrulhar o envelope e montar o pacote do motor)	do contrato de entrada declarado na tarefa
uma conversão de saída por tarefa (encaixar o resultado no estado acumulado)	do contrato de saída declarado na tarefa
a lista de tarefas suportadas e a apresentação do servidor	da sequência de tarefas
a ordem e as dependências	da rede de Petri

Isso é **geração de código nova** — não é cópia, não é conserto e não é trabalho por projeto. É a peça que hoje não existe e sem a qual **nenhuma** aplicação gerada executa no mesmo esquema do Tropical.

Parte 4 — Interface A: a Bancada de Execução dentro do LangNet

O que é: a tela da 116, trazida integralmente, como **etapa nova do pipeline**. É a nossa ferramenta de diagnóstico — o cliente nunca a vê.

Onde entra: etapa **Execução**, depois de Código, seguindo o padrão obrigatório das nossas etapas — seleção de origem e versão, executar, refinar por conversa, aprovar. Ela lê a rede do próprio projeto: mesmo banco, mesmo registro, nada novo.

O que traz da 116, sem reescrever: a tela grande com as seis abas, o painel móvel, o painel de etiquetas, o desenho da rede, o painel do simulador, o juiz de lugar concluído e — principalmente — a dupla que intercepta a conversa e mantém a ligação única com fila e cache.

O que precisa ser adaptado:

1. **Tirar os nomes da Tropical de dentro do painel móvel.** Ele tem de receber a lista de tarefas do projeto, não conhecê-la de cor.
2. **Descobrir a porta do servidor de agentes pelo projeto.** Hoje a fábrica grava a porta dentro do código de cada lugar, mas a tela também precisa dela para mostrar ligado/desligado.
3. **Descartar o que está morto** — a tela antiga de 23 mil linhas, a tela alternativa e as quatro abas-maquete não vêm.

O que tem de ser corrigido antes, senão a bancada não mostra nada: os Defeitos 1, 2 e 3 — nesta ordem. E os três são corrigidos **na fábrica**, nunca no artefato.

Parte 5 — Interface B: o padrão da aplicação final

Pergunta a responder: no Uso do Solo, no Quântica, no BioByte — onde fica a interface do sistema de agentes?

Resposta: não fica numa tela separada. Fica dentro da tela de negócio do caso de uso que o fluxo atende. A bancada é nossa; o cliente nunca vê rede de Petri, nem lugar, nem transição.

E a mudança de fundo: **a tela de negócio não chama tarefa — ela dispara um fluxo.** Hoje ela diz "execute a tarefa classificar". Passa a dizer "execute o fluxo de análise com estes dados" e acompanha. Quem decide a ordem é a rede, não a tela.

As quatro peças que o gerador passa a emitir

1. Ação agêntica. O botão que o operador aperta, com nome de negócio — "Analisar conformidade", "Gerar laudo", "Classificar mensagens". Leva o que está no formulário e dispara o fluxo pelo seu ponto de entrada.

2. Faixa de andamento. A sequência de passos do fluxo em uma linha, cada um acendendo conforme acontece, com o nome de negócio de cada etapa. É a rede traduzida para a linguagem do operador.

3. Painel de evidência. Recolhido por padrão, aberto quando o operador quiser: o que o agente consultou, com que argumentos, o que a fonte respondeu e como concluiu. É o que dá confiança e é o que audita. **Alimentado pelas mesmas etiquetas da bancada** — a mesma instrumentação serve aos dois lados.

4. Ponto de aprovação. Quando o fluxo precisa de gente — aprovar o laudo, confirmar o pedido — a rede para numa transição que espera e a tela mostra o que decidir.

Isto já existe na 116 e eu não tinha visto: o modo "Pausar por Tarefa" suspende a execução após cada lugar e aguarda. A peça está pronta para ser generalizada.

O contrato entre a tela e o servidor de agentes

Quatro verbos, no lugar do "execute esta tarefa" de hoje:

verbo	o que faz
iniciar fluxo	dispara pelo ponto de entrada, com os dados do formulário
acompanhar	recebe os passos, as etiquetas e o avanço
responder à aprovação	devolve a decisão do operador e libera a transição que esperava
resultado	entrega o resultado final com os campos já no vocabulário da tela

O desembulho do envelope continua sendo trabalho do tradutor do lado do servidor, como na 116.

E a regra inegociável

Falha é falha. Se um passo não conseguiu fazer o que tinha de fazer — o e-mail não saiu, a consulta não respondeu — o fluxo **para ali**, a faixa fica vermelha naquele passo e a tela diz o que falhou. Nunca segue verde com o dado vazio.

Parte 6 — O back-end genérico e o banco

O que ele faz hoje: serve a lista de projetos e o projeto individual, devolvendo a rede de Petri gravada; aceita criar, alterar e apagar projeto; e oferece **um conjunto de portas para registrar execução** que ninguém nunca usou.

O que já temos no LangNet: o mesmo banco e as mesmas três tabelas. Ou seja, **não precisamos de back-end novo** — precisamos passar a escrever no que já existe.

As três decisões desta parte:

1. **Separar o significado da tabela de sessões.** Ela foi desenhada para execução de rede e o nosso pipeline de geração passou a ocupá-la. Ou o pipeline migra para tabela própria, ou a execução ganha a sua. Sem isso, duas coisas diferentes disputam o mesmo registro.
 2. **Passar a gravar de verdade.** Ao fim de cada rodada: uma linha de sessão com marcação inicial e final e os contadores; uma linha por tarefa com entrada, saída e registro; e uma linha por documento produzido. **Hoje isso é zero em tudo.** É o que permite comparar rodadas, provar regressão e auditar.
 3. **Quem grava é a bancada, não o servidor de agentes.** O servidor executa e relata; quem tem a visão da rodada inteira é a tela. Assim o mesmo servidor serve à bancada e à aplicação final sem saber de banco.
-

Regra de passagem — nenhuma etapa avança sem prova

Nenhum passo deste plano é dado por concluído sem caso de teste que comprove.

A regra, definida pelo usuário em 25/09/2026:

1. Antes de corrigir, **escrever o caso de teste que reproduz o defeito** e rodá-lo, mostrando a falha. Sem isso não se sabe o que se está consertando.
2. Corrigir.
3. Rodar de novo e mostrar a passagem.
4. **Provar contra o sistema de verdade** — não basta teste de unidade: a prova final é a lógica real do projeto rodando contra o servidor de agentes real.
5. Só então passar ao próximo.

Os casos de teste ficam em `tests/execucao/` e são **cumulativos**: cada passo novo roda também os dos passos anteriores, para que uma correção não desfaça outra.

Registro das provas

passo	caso de teste	resultado
1 — destravar o executor	prova-lugar-espera.mjs (5 casos: sem espera, espera simples, espera de conversa, laço por predecessor, falha)	antes 1 de 5 ; depois 5 de 5
1 — prova fim-a-fim	prova-logica-real.mjs — lógica real do lugar, nosso processador, servidor de agentes real	passou em 43,2s , envelope completo, status: completed

Achado da prova fim-a-fim: a tarefa levou **43,2 segundos**. O prazo original escrito no lugar era de **30 segundos** — ou seja, a lógica de referência estoura nesta máquina. Confirma o dimensionamento que a nossa fábrica já faz (60s no caso comum, 180s para classificar, analisar, buscar e extrair) e reforça o Defeito 3: quando estoura, tem de reclamar, não seguir calado.

Parte 7 — Ordem de trabalho

#	passo	por quê nesta ordem
1	Destravar o executor (Defeito 1)	é uma linha e sem ele nada pode ser testado
2	Trazar o juiz de lugar concluído (Defeito 2)	a rede precisa parar de disparar no vazio
3	Tratar erro como erro (Defeito 3)	enquanto falha passar calada, todo teste mente
4	Igualar o protocolo do nosso servidor (Parte 3-B)	handshake de abertura, fluxo passo a passo, etiquetas e o mesmo envelope — sem isto a tela trazida mostra telas vazias
5	Trazar as 9 peças que faltam (Parte 3-B)	com destaque para a ligação única com fila e cache, e o juiz de lugar concluído
6	Bancada de Execução como etapa nova	validar rodando o TropicalSales pela nossa tela — mesmo projeto, mesmo banco, o resultado tem de bater com o da 116
7	Etapla nova: gerar o tradutor	derivar o estado padronizado e as conversões a partir dos artefatos do projeto — vale para toda aplicação gerada, não para um caso
8	Gravar a execução no banco (Parte 6)	a partir daqui cada rodada deixa rastro
9	Provar a generalidade em dois casos-teste	BioByte e Uso do Solo são instâncias de prova da fábrica, não destinos do trabalho
10	As quatro peças da aplicação final (Parte 5)	trocar "chamar tarefa" por "disparar fluxo" no gerador de telas

Os passos 1 a 3 são conserto de uma linha cada. Os passos 4 a 6 são **transferência** — o que deveria ter sido feito antes e não foi. O 7 é geração de código nova. O 10 é mudança de arquitetura e deve vir **depois** que a bancada estiver mostrando a verdade — senão mudamos no escuro.

Anexo — o que copiar da 116

Vem integralmente: a tela grande com as seis abas; o painel móvel; o painel de etiquetas; o desenho da rede; o painel do simulador; o juiz de lugar concluído; o motor genérico de execução; e a dupla interceptador + cliente central.

Vem adaptado: a moldura com a barra lateral (o LangNet já tem a sua navegação).

Não vem: a tela antiga de 23.625 linhas; a tela alternativa; as quatro abas-maquete; o seletor de projeto alternativo.

Corrigir ao trazer: os nomes de tarefa da Tropical gravados dentro do painel móvel, e os endereços de serviço gravados no código.